

SATRF scoring workflow (current system)

This document describes how match scores enter the website, how they are stored, validated, and how members see rankings. It reflects the production stack as of the ISSF Firestore implementation (Next.js API routes + Firebase Admin), not the retired Python backend.

1. Overview

Role | What they do

****Admin**** | Creates official scores via manual entry, Excel import, or PDF import. Manages the scores list (status, soft-delete).

****Member**** | Cannot submit official scores. Views own results (`/scores` + API) and public rankings.

****System**** | All writes go through `validateScoreInput()` != `buildScore()` != Firestore `scores` collection. Totals are ****computed server-side****, not trusted from the client.

Ranking metric: For each shooter, the leaderboard uses the average of decimalTotal across their official scores in a discipline (and optional category filter). Higher average = better rank.

2. Score data model

2.1 Firestore collection: `scores`

Each document is one completed match card for one shooter.

Field | Description

`userId` | Firebase UID of the member, or `null` if guest / not linked

`shooterName`, `club` | Display + matching for imports

`category` | `open` \ | `junior` \ | `veteran` \ | `ladies`

`eventId`, `eventName`, `date` | Match context (`date` = `YYYY-MM-DD`)

`discipline` | `prone\_50m` \ | `three\_position\_50m`

`scoringType` | Usually `decimal`

`positions[]` | Position blocks (see below)

`decimalTotal`, `integerTotal`, `innerTens`, `totalShots` | ****Derived**** on save

`status` | `official` (counts for rankings) or `provisional`

`source` | `manual` \ | `excel` \ | `pdf`

`createdBy`, `createdAt`, `updatedAt` | Audit

`deleted` | Soft-delete flag (excluded from lists when set)

2.2 Position blocks and series

```
positions: [
  {
    position: "prone",          // or kneeling / standing for 3P
    series: [
      { seriesNumber: 1, decimal: 100.7, integer: 95, innerTens?: 4, shots?: [...] }
      ...
    ],
    decimalTotal, integerTotal, innerTens  // per-position totals (derived)
  }
]
```

2.3 ISSF discipline rules (code: `src/lib/issf.ts`)

Discipline | Positions | Series per position | Shots | Max decimal / series | Max match decimal

`prone\_50m` | prone only | 6 | 60 | 109.0 | 654.0

`three\_position\_50m` | kneeling != prone != standing | 4 each | 120 | 109.0 | 1308.0

- Decimal scoring: up to 10.9 per shot; series cap 109.0.
- Official saves require the full series count per position (strict validation).
- Provisional allows incomplete series (warnings only).

Types live in `src/types/scores.ts`. Validation and totals: `validateScoreInput()`, `buildScore()` in `src/lib/issf.ts`.

3. Security and access

3.1 Firestore rules (`firestore.rules`)

- scores: read = public (leaderboard). write = admins only (client SDK). Members cannot self-insert scores.
- Actual admin writes use the Admin SDK in API routes (bypasses client rules).

3.2 API authentication

- Admin routes: Authorization: Bearer <Firebase ID token>, verified via `verifyAdminFromToken()` (`src/lib/admin.ts`).
- Member route GET `/api/scores/my-scores`: token must match `userId` on returned documents.

3.3 Admin UI access

- Pages under `/admin/*` require admin role + middleware auth cookies.
- Score entry hub: `/admin/scores/import`.

4. Admin entry paths (detailed)

All three paths end at the same write pipeline:

```
ScoreInput[]  !' POST /api/admin/scores (or /api/admin/scores/import for Excel)
               !' validateScoreInput (per row)
               !' optional findMemberUid(name + club)
               !' buildScore()
               !' Firestore batch write
```

4.1 Manual entry

UI: `/admin/scores/import` !' tab Manual Entry (`ManualEntryComponent.tsx`)

Flow:

1. Choose discipline (prone or 3P — UI supports both).
2. Choose event from dropdown (or custom event name).
3. Choose member or enter guest name + club.
4. Enter category, match date, and per-series decimal (+ optional integer) for each position.
5. Add to batch for multiple shooters on the same match setup.
6. Save !' POST `/api/admin/scores` with `{ scores: ScoreInput[] }`.

Linking: If a member is selected, `userId` is sent. Otherwise server tries `firstName` + `lastName` + `club` against users.

Source tag: manual.

4.2 Excel import (prone bulk)

UI: `/admin/scores/import` !' Upload Excel/CSV

Template: `/admin/scores/template` !' downloads SATRF_Prone_50m_Score_Import.xlsx

Sheet to fill: Prone Scores (one row per shooter per match)

Column	Required	Notes
Date (YYYY-MM-DD)	Yes	Excel date cells OK
Event Name	Yes	Text label (not auto-linked to event ID)
Discipline	Yes	<code>`prone_50m`</code> for current template
Shooter Name	Yes	Should match member profile for linking
Club	Yes	Must match member <code>`club`</code> for auto-link
Category	Yes	<code>`open`</code> , <code>`junior`</code> , <code>`veteran`</code> , <code>`ladies`</code>
S1 Decimal ... S6 Decimal	Yes	Series totals (0–109.0)
S1 Integer ... S6 Integer	No	Ring totals from target PDF
Status	No	<code>`official`</code> (default) or <code>`provisional`</code>
Notes	No	Not imported

Flow:

1. Upload .xlsx / .xls / .csv.
2. Client parses sheet !' `src/lib/excelImport.ts` validates rows.
3. Preview shows valid / error rows.
4. Import !' POST `/api/admin/scores/import` with `ScoreInput[]` (source: excel).
5. Same server validation and Firestore write as manual entry.

Not in template yet: `three_position_50m` multi-position columns (3P is manual entry only for now).

4.3 PDF import (prone, electronic target)

UI: /admin/scores/import !' tab PDF Import

Important: Member, event, date, and category are chosen in the UI — not read from the PDF.

Step A — Analyse PDF

POST /api/admin/scores/parse-pdf

- Body: { reportType: "summary" | "target", pdfBase64: "..." }
- Server extracts text with pdf-parse v1, then:

Report type | Parser | What it reads

Summary | parseSummary.ts | Lines S1–S6 totals only (first block before series 7–12). Handles compact text e.g. S1100.7 + 495 (inner + integer).

Target | parseTarget.ts | Blocks SERIES N ... TOTAL : decimal (integer) for series 1–6 only.

- 80-shot practice PDFs: series 7+ are ignored; info warning only.
- Requires 6 series before save.

Step B — Save score

POST /api/admin/scores with one ScoreInput built by parsedPdfToScoreInput() (source: pdf, discipline: prone_50m, userId from selected member).

4.4 Admin scores management

List: /admin/scores

- GET /api/admin/scores — filter by status, discipline, search (name/club/event).
- Edit status (official / provisional).
- Delete via DELETE /api/admin/scores/[id] (soft-delete).

5. Member-facing views

5.1 My scores

- API: GET /api/scores/my-scores?discipline=...
- Returns documents where userId = logged-in member, not deleted, newest first.
- Used by member dashboard / profile flows that call this API.

5.2 Rankings / leaderboard

Public API: GET /api/leaderboard/overall

Query params:

- discipline (default prone_50m)
- category (optional: open, junior, veteran, ladies)

Logic:

1. Load all official scores for discipline (+ category if set), not deleted.
2. Group by userId, or shooterName|club if no UID.
3. Per group: average of decimalTotal, best single match, eventCount.
4. Sort by average descending, assign rank.

Pages:

- /scores — public rankings table (same API).
- /scores/leaderboard — richer UI (may call leaderboardAPI in src/lib/api.ts; overall endpoint is the source of truth for ISSF averages).
- /admin/rankings — admin view of the same data.

6. End-to-end diagram (prone match)

(Flow diagram — see SCORING_WORKFLOW.md in the repo for the full chart.)

7. What members cannot do

- Submit official ISSF series scores through the app (Firestore blocks non-admin writes).
- /scores/upload is a legacy single-number upload form (scoresAPI) — not the ISSF series model. Official results should use admin import only.

8. Operational checklist

After importing a prone score:

1. Admin Scores — row appears with correct decimalTotal, source, userId.
2. Member My scores — visible if userId was set (member selected in PDF/manual, or name+club match on Excel).
3. Rankings — counts only if status === official and discipline/category filters match.

PDF troubleshooting:

- Use Summary tab for summary PDFs, Target tab for target/match PDFs.
- File must be a real PDF (%PDF header), not a renamed export.
- Prone match = 6 series; extra series on 80-shot cards are ignored.

Excel troubleshooting:

- Use exact header row from template.
- Shooter name + club must match Firestore users for automatic userId linking.

9. Key source files

Area | Path
 Types | `src/types/scores.ts`
 ISSF rules & totals | `src/lib/issf.ts`
 Excel mapping | `src/lib/excelImport.ts`
 PDF parsers | `src/lib/pdfImport/\*`
 Write API | `src/pages/api/admin/scores.ts`
 Excel import API | `src/pages/api/admin/scores/import.ts`
 PDF parse API | `src/pages/api/admin/scores/parse-pdf.ts`
 Import UI | `src/pages/admin/scores/import.tsx`
 Template download | `src/pages/admin/scores/template.tsx`
 Leaderboard API | `src/pages/api/leaderboard/overall.ts`
 My scores API | `src/pages/api/scores/my-scores.ts`
 Firestore rules | `firestore.rules`

10. Planned / not yet built

- 3P Excel template and import columns for kneeling / prone / standing series.
- Per-shot storage from target PDFs (only series totals today).
- PDF !' event ID auto-link (event is manual dropdown).
- PDF !' member auto-match (member is manual dropdown).
- Deeper admin statistics dashboard (beyond rankings table).

Document version: aligned with master branch ISSF scoring implementation.